

AgentLogs: A Dataset for Opening the Black Box of GitHub’s Cloud Agent

Jonan Richards*, Kosei Horikawa†, Youmei Fan†, Yutaro Kashiwa† and Mairieli Wessel*

*Radboud University, the Netherlands Email: {jonan.richards,mairieli.wessel}@ru.nl

†Nara Institute of Science and Technology, Japan Email: {horikawa.kosei.hk1,fan.youmei.fs2,yutaro.kashiwa}@naist.ac.jp

Abstract—Generative AI-based software engineering agents are becoming routine contributors to real-world software projects. On GitHub, developers can assign tasks to the Copilot cloud agent, which autonomously explores the repository, edits code, runs commands, and opens or reviews pull requests, producing a detailed log of every step along the way. While existing datasets capture outcomes of agent contributions, such as agent-authored pull requests, the process by which agents produce these contributions remains largely unexplored. To address this gap, we introduce **AgentLogs**, a large-scale dataset of agent activity on GitHub. **AgentLogs** comprises 307,416 agent tasks and 549,239 agent sessions in 35,810 of the 1,812,362 popular public repositories that we scanned, together with 64,255,174 session log entries that record each agent run step by step, including prompts, intermediate reasoning, tool calls (e.g., file edits, git operations, and GitHub interactions), and token usage. By exposing not only what agents contribute but also how they work, **AgentLogs** enables research on agent behavior, efficiency and cost, task formulation, failure modes, and human-agent collaboration in agentic software engineering.

Index Terms—AI software engineering agents, GitHub Copilot, agent logs, pull requests, mining software repositories, dataset

I. DATASET OVERVIEW

GitHub allows developers to employ autonomous agents to perform software engineering tasks [1], [2]. Developers assign a *task*, potentially with a natural-language prompt, from e.g., the Agents tab on GitHub.com (see Figure 1), in a comment on a pull request, as an automation, or on a connected third-party tool such as Jira, Linear, or Slack [2], [3]. GitHub Copilot cloud agent then works asynchronously in a temporary environment, exploring the repository, planning

and implementing changes, editing files, running commands, and interacting with GitHub by, e.g., reading, reviewing, and creating issues, pull requests, and comments [2]. Tasks can also use repository-defined custom agents or third-party models such as Claude and Codex, configured on top of the Copilot agents platform [1], [4]. Each task can run one or more *sessions*, each of which produces a detailed *session log* recording model output, tool calls, usage metadata, prompts, and reasoning traces (Figure 1).

We seeded our data collection process using an up-to-date dataset of GitHub repositories with over 10 stars [5], accessed at <https://seart-ghs.si.usi.ch/> on July 10, 2026. Next, we used the GitHub REST API to fetch agent task identifiers for each repository. Although the REST API has an endpoint to fetch task and session data from these task identifiers, this data is limited. Instead, we used undocumented endpoints of the GitHub Copilot API (<https://api.githubcopilot.com>) to fetch task and sessions metadata for each task. This use of undocumented API endpoints is allowed: the GitHub Acceptable Use Policies explicitly allow any method of obtaining publicly available data [6], and we respected rate limits per the Terms of Service [7]. As there is no documented API endpoint to fetch session logs, we fetched these using the same undocumented API, then parsed the logs as lines of JSON. Task and session records often include identifiers for related GitHub resources, such as pull requests, branches, and workflow runs. However, we did not fetch further metadata for these resources for this version of the dataset. All data was collected between July 10 and July 17, 2026.

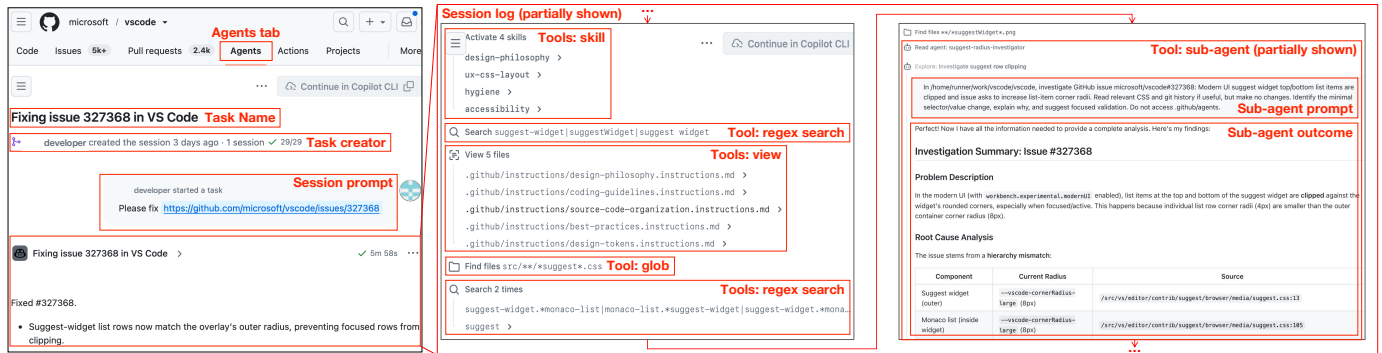


Fig. 1. Example of an agent task with a single session on GitHub. The session overview shows the prompt and the resulting summary (left), which can be expanded to show a session log detailing tool calls and outputs such as file searches (center) and delegation to sub-agents (right). Note that our dataset captures session logs in more detail than currently displayed on the GitHub website.

TABLE I
SUMMARY OF THE AGENTLOGS DATASET

	# Records	Size	Table	Content
Repositories – 1.98% with agent tasks	1,812,362 35,810	395.6 MB	<code>repositories</code>	Public GitHub repositories with over 10 stars (metadata including name, license, language, stars, forks, timestamps, labels, topics).
Agent tasks	307,416	77.1 MB	<code>agent_tasks</code>	Agent assignment on a repository (metadata including name, request, state, creator, timestamps, branch/PR identifiers).
Agent sessions	549,239	225.1 MB	<code>agent_sessions</code>	Agent runs within a task (metadata including model, prompt, outcome, usage, and branch/PR identifier for that session).
Log entries	64,255,174	56.0 GB	<code>agent_session_logs</code>	Session log events (including messages, usage details, tool calls for file edits, git, and GitHub issues, PRs, comments, CI).
Users	33,573	39.7 MB	<code>users</code>	Users related to the agent tasks and sessions (only GitHub id and username).
Total	66,957,764	56.7 GB		

II. INTERNAL STRUCTURE

A summary of the size, number of records, and data descriptions in the AgentLogs dataset is shown in Table I. Note that, although the dataset includes a `users` table, their GitHub IDs and usernames were extracted from agent task and session metadata without further user data being fetched from the API. The repository, agent task, agent session, log entry, and user records are stored in Parquet tables. These tables have been sharded to enable easier downloading and parallel processing, in chunks of at most ~ 220 MB for log entries, ~ 120 MB for sessions, and ~ 80 MB for the remaining tables. The AgentLogs repository documentation (section III) includes a table relationship diagram and the full data schema mapped to GitHub and Copilot API fields. See Appendix A for a partial session log, showcasing the depth of the data captured in AgentLogs.

III. HOW TO ACCESS

The AgentLogs dataset is available on Hugging Face at <https://huggingface.co/datasets/risenlab/agentlogs> (version 0.2). Schema documentation, example analysis notebooks, and a sample of the data are available on GitHub at <https://github.com/risenlab/agentlogs>. Python type definitions for every table are available as a PyPI package, `risenlab-agentlogs`. Code is licensed under MIT; the dataset under CC BY 4.0.

Given the scale of the dataset, we provide example analysis scripts that demonstrate practical ways to query and process the data efficiently. The examples cover DuckDB for SQL over Parquet, Polars for lazy DataFrame-style aggregation across the full dataset, a streaming approach that reads Parquet records incrementally for low-memory exploration with type support and custom Python analysis (e.g., regex parsing and sequence analysis), and Hugging Face Datasets streaming from the Hub without a local copy. Researchers can use these or other tools, depending on their research questions and preferred workflow.

The GitHub repository includes a small sample of the full dataset, containing 3 repositories, 4 agent tasks, 5 agent sessions, 416 log entries, and 3 users.

IV. POTENTIAL RESEARCH QUESTIONS

Unlike outcome-level datasets of agent contributions, AgentLogs captures a step-by-step execution trace of agent runs. We outline example research questions below:

1) Agent Behavior and Strategies:

- What workflow patterns (e.g., exploring, editing, testing) do agents follow across sessions, and how do these patterns differ between task types such as bug fixes and feature requests?
- How do agents make use of repository provided context (e.g., custom instructions and skills), and is such context associated with different behavior and outcomes?

2) Human-Agent Collaboration:

- How do agents respond to reviewer feedback in follow-up sessions, and which types of review comments are hardest for agents to address?
- How does developers’ usage of agents evolve over time as they gain experience (e.g., changes in request style, task scope, and follow-up behavior)?

3) Adoption and Ecosystem:

- What characterizes the 1.98% of popular repositories that adopt coding agents?
- How does the behavior of the underlying models compare (sessions include model metadata), and how has agent usage grown over time?

REFERENCES

- [1] GitHub, “GitHub Copilot · Agents on GitHub,” <https://github.com/features/copilot/agents>, accessed: 2026-07-28.
- [2] GitHub Docs, “About GitHub Copilot cloud agent,” <https://docs.github.com/en/copilot/concepts/agents/cloud-agent/about-cloud-agent>, accessed: 2026-07-28.
- [3] —, “About Copilot integrations,” <https://docs.github.com/en/copilot/concepts/tools/about-copilot-integrations>, accessed: 2026-07-28.
- [4] —, “About custom agents,” <https://docs.github.com/en/copilot/concepts/agents/cloud-agent/about-custom-agents>, accessed: 2026-07-28.
- [5] O. Dabic, E. Aghajani, and G. Bavota, “Sampling projects in github for MSR studies,” in *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. IEEE, 2021, pp. 560–564.
- [6] GitHub Docs, “GitHub Acceptable Use Policies,” <https://docs-internal.github.com/en/site-policy/acceptable-use-policies/github-acceptable-use-policies>, accessed: 2026-07-28.
- [7] —, “GitHub Terms of Service,” <https://docs-internal.github.com/en/site-policy/github-terms/github-terms-of-service>, accessed: 2026-07-28.

APPENDIX A

EXAMPLE SESSION LOG

Below are shown several log entries for a single session (id: 00bfa1b1-48e7-41b4-b57c-0b6ab7114905). Uninformative properties (including those with null values) have been omitted for readability, and escaped newline characters in strings (\n) have been converted into line breaks. Usernames and repository names have been redacted.

```
...
{
  "entry_index": 6,
  "data": {
    "content": [{
      "type": "text",
      "text": "You have been given comments on the previous commits you made in the repository.

You are working on an issue in the '[owner/repo]' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/[repo]/[repo] (not in /tmp/inputs
). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-04-15T17:46:17.915Z</current_datetime>

Consider the following problem statement:

<problem_statement>

----
*This section includes details on the pull request with code changes you have already submitted
to fix the problem above.*

<pr_title>
Use long curl option names and sort check_dep calls alphabetically
</pr_title>

<pr_description>
Replaces curl short flags with long equivalents (compatible with both Linux and macOS) and
fixes the alphabetical ordering of 'check_dep' calls in 'install.sh'.

- **'lib/core_utils.sh':** '-sL ... -o' \u2192 '--silent --location ... --output'
- **'install.sh':** moved 'check_dep \"curl\"' from the bottom of the list to its correct
alphabetical position (between 'cp' and 'cut')
</pr_description>

## Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>[@user]</author>
@copilot+claude-sonnet-4.6 Update changelog
</comment_old>

<comment_old>
<author>@copilot</author>

&gt; @copilot+claude-sonnet-4.6 Update changelog

Done in 9350c7a \u2014 added the unreleased changes to 'CHANGELOG.md' under the '## Unreleased'
section.
</comment_old>

</pr_comments>
```

```

<pr_reviews>

<comment_thread>
<file>CHANGELOG.md:10</file>
<original_commit>9350c7a</original_commit>

<comment_new>
<comment_id>3088356364</comment_id>
<author>[@user]</author>
@copilot Only one item should be added in this section i.e. \"Replaced `wget` with `curl` for
self upgrade.\"
</comment_new>

</comment_thread>

</pr_reviews>

</comments>

----
The last **4** git commits in this branch are the changes you have made so far. Use those as
your change commit history.
</problem_statement>

[ ... further system prompt instructions to copilot ... ]"
  }},
  "role": "user"
}
}

{
  "entry_index": 7,
  "data": {
    "choices": [
      {
        "delta": {
          "reasoning_text": "The reviewer wants only one changelog entry: \"Replaced `wget`
with `curl` for self upgrade.\" Let me update the CHANGELOG.md accordingly.",
          "role": "assistant",
        },
      },
      {
        "delta": {
          "role": "assistant",
          "tool_calls": [{
            "function_name": "edit",
            "function_arguments": "{ \"path\": \"\n/home/runner/work/[repo]/[repo]/CHANGELOG.md
\", \"old_str\": \"## Unreleased\n\nChanged:\n\n* Replace `wget` with `curl` for downloading
files.\n* Use long option names for `curl` command for clarity.\n* Keep `check_dep` calls
alphabetically ordered in `install.sh`.\", \"new_str\": \"## Unreleased\n\nChanged:\n\n*
Replaced `wget` with `curl` for self upgrade.\" }",
            "type": "function"
          }
        ]
      },
    ],
  },
  "created": "2026-04-15 17:46:26",
  "model": "claude-sonnet-4.6",
  "usage": {
    "completion_tokens": 226,
    "prompt_tokens": 33010,
    "prompt_tokens_details": {
      "cached_tokens": 0,
    },
    "total_tokens": 33236
  }
}

```

```

    },
  }
  {
    "entry_index": 8,
    "data": {
      "content": [{
        "type": "text",
        "text": "File /home/runner/work/[repo]/[repo]/CHANGELOG.md updated with changes.",
      }],
      "role": "tool",
    }
  }
  ...
  {
    "entry_index": 13,
    "data": {
      "choices": [{
        "delta": {
          "role": "assistant",
          "tool_calls": [{
            "function_name": "reply_to_comment",
            "function_arguments": "{\n\"comment_id\": \"3088356364\", \"reply\": \"Fixed in a982039 \\\nu2014 the `## Unreleased` section now contains only the single entry: \\\"Replaced `wget` with `curl` for self upgrade.\\\"\", \"type\": \"function\""}
          }
        },
      }],
      "created": "2026-04-15 17:46:34",
      "model": "claude-sonnet-4.6",
      "usage": {
        "completion_tokens": 113,
        "prompt_tokens": 33750,
        "prompt_tokens_details": {
          "cached_tokens": 33268,
        },
        "total_tokens": 33863
      }
    }
  }
  {
    "entry_index": 14,
    "data": {
      "content": [{
        "type": "text",
        "text": "Replied to comment_id 3088356364. If you are done responding to comments, make sure to write a summary of what you did before wrapping up.",
      }],
      "role": "tool",
    }
  },
}
  ...

```